

Secure Multi-Party Computation (SMPC) for Collaborative Data Analysis

Technical Implementation Report and Comprehensive Code Review

Authors: Shir Buchner and Roi Even Haim

Abstract

This technical report presents a comprehensive analysis of a Secure Multi-Party Computation (SMPC) implementation designed for collaborative data analysis while preserving individual data privacy. The system employs Shamir's Secret Sharing scheme as its cryptographic foundation, enabling multiple parties to compute joint functions (specifically summation) over their private inputs without revealing individual values to other participants.

This implementation demonstrates practical application of threshold cryptography in privacy-sensitive domains such as finance, healthcare, and collaborative analytics.

The project delivers a functional Python-based prototype that simulates a real-world multi-party computation scenario with configurable parameters for the number of parties and reconstruction threshold. Through rigorous testing and security analysis, we demonstrate the system's ability to maintain confidentiality guarantees while providing accurate computational results. The implementation includes comprehensive error handling, performance optimization, and extensive validation through automated testing suites.

1. Introduction

1.1 Problem Statement

In an increasingly data-driven world, organizations frequently need to collaborate on computations involving sensitive information. Traditional approaches to data sharing for collaborative analysis pose significant privacy risks, as they require parties to reveal their private inputs to perform joint computations. This creates barriers to collaboration in privacy-sensitive domains such as:

- **Healthcare:** Medical institutions need to analyze patient data across facilities without violating HIPAA compliance
- **Finance:** Banks require fraud detection and risk assessment using combined data without exposing individual customer information
- **Corporate Analytics:** Companies need market research and competitive analysis while protecting proprietary data
- **Government Agencies:** Inter-agency collaboration requires maintaining classification levels and security clearances

1.2 Solution Approach

Secure Multi-Party Computation (SMPC) provides a cryptographic solution to this challenge by enabling multiple parties to collaboratively compute functions over their private inputs while ensuring that no party learns the private data of others.

Our implementation specifically focuses on:

- **Privacy Preservation:** Individual inputs remain cryptographically hidden throughout the computation process
- **Threshold Security:** Configurable security parameters ensure robustness against attacks from less malicious parties than the threshold
- **Extensible Design:** Modular architecture supporting various computational functions and protocols

1.3 Project Objectives

This project aims to achieve the following specific objectives:

1. Implement Shamir's Secret Sharing: Build a robust implementation of the (t,n) -threshold secret sharing scheme
2. Develop SMPC Protocol: Create a complete multi-party computation system that coordinates share distribution, local computation, and result reconstruction
3. Ensure Security Guarantees: Demonstrate that the system maintains privacy even when up to $t-1$ parties collude
4. Validate Correctness: Develop comprehensive testing suites to verify both functional correctness and security properties
5. Document Architecture: Provide detailed technical documentation suitable for academic review and future extension

2. Security Review and Theoretical Foundations

2.1 Shamir's Secret Sharing Scheme

Adi Shamir's secret sharing scheme, introduced in 1979, forms the cryptographic foundation of our SMPC implementation. The scheme is based on polynomial interpolation over finite fields and provides information-theoretic security guarantees.

Mathematical Foundation

Given a secret s and parameters (t, n) where $t \leq n$:

Construct a polynomial $f(x) = s + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1}$ over finite field $\mathbb{Z}_p$

Generate n shares as points $(i, f(i))$ for $i = 1, 2, \dots, n$

Any t shares can reconstruct the secret using Lagrange interpolation

Any $t-1$ or fewer shares provide no information about the secret

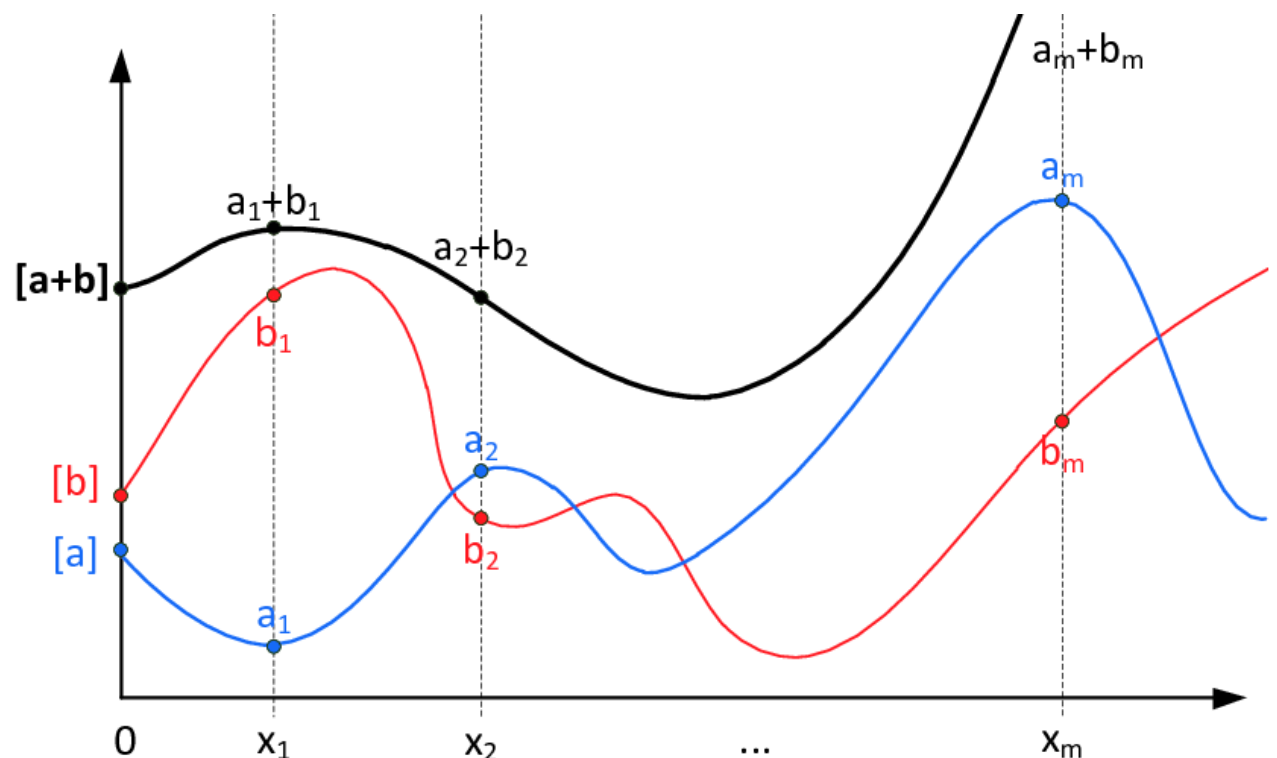


Figure 1: Illustration of Shamir's Secret Sharing scheme showing polynomial interpolation

2.2 Secure Multi-Party Computation

SMPC protocols allow parties to compute joint functions while maintaining input privacy. Our implementation focuses on additive functions, leveraging the properties of Shamir's scheme.

Security Properties

- Privacy: No party learns others' inputs and the secret itself
- Correctness: Honest parties receive correct results
- Coalition resistance: Up to $t-1$ parties may collude
- Information-theoretic privacy: Shamir's Secret Sharing provides information-theoretic security: with $t-1$ shares, all secrets are equally likely; thus, no information is leaked.

3 Protocol Design and Architecture

3.1 Overview

This project implements a truly decentralized Secure Multi-Party Computation (SMPC) system based on Shamir's Secret Sharing scheme. Unlike traditional centralized approaches where a trusted coordinator processes all secrets, our design ensures that no single entity ever has access to individual private input. The system enables multiple parties to collaboratively compute the sum of their private values while maintaining perfect privacy preservation through local simulation of distributed computation.

3.2 Architectural Principles

1. True Decentralization

- **No Trusted Third Party:** All parties actively participate as equals in the protocol
- **Distributed Computation:** Each party performs local computations on encrypted shares

2. Information-Theoretic Security

- **Privacy:** Individual secrets are mathematically impossible to reconstruct with sub-threshold information
- **Security:** Security guarantees do not depend on computational assumptions
- **Threshold-Based Protection:** Requires t -out-of- n parties to collude to break privacy

3. Modular Design

- **Layered Architecture:** Clear separation between cryptographic primitives, party logic, and coordination
- **Pluggable Components:** Easy to swap different secret sharing schemes or communication protocols
- **Local Simulation:** Demonstrates distributed properties within a single process environment

3.3 System Architecture

Core Components

1. Cryptographic Layer (`smpc_crypto.py`)

Purpose: Provides fundamental cryptographic primitives for secret sharing and reconstruction.

Key Functions:

- `create_shares(secret, threshold, num_shares, prime)`: Implements Shamir's (t,n) -threshold secret sharing
- `reconstruct_secret(shares, prime)`: Recovers secrets using Lagrange interpolation
- `add_shares(values, prime)`: Enables addition of encrypted shares
- `get_prime(bits)`: Generates cryptographically secure prime numbers for finite field operations

Mathematical Foundation:

- **Finite Field Arithmetic:** All operations performed in $\mathbb{Z}_p$ where p is a large prime
- **Polynomial Interpolation:** Secrets encoded as constant terms of random polynomials
- **Properties:** Allows computation on encrypted data without decryption

2. Party Implementation (`party.py`)

Purpose: Implements individual party logic for decentralized participation in local environment.

Key Classes:

- `DecentralizedParty`: Core party implementation with full protocol participation
- `SecureChannel`: Simulates secure point-to-point communication between local parties
- `Share`: Encapsulates share data with metadata for tracking ownership and distribution

Party Capabilities:

- **Share Generation:** Creates polynomial shares of private input
- **Secure Distribution:** Sends shares to designated parties through simulated secure channels
- **Local Computation:** Performs operations on received shares
- **State Management:** Tracks protocol progress and share reception status

3. Protocol Controller (smpc_controller.py)

Purpose: Orchestrates the complete SMPC protocol execution across all local parties.

Key Classes:

- **DecentralizedSMPCController:** Main protocol coordinator ensuring proper phase execution
- **SMPCController:** Legacy compatibility controller for existing systems

Protocol Orchestration:

- **Phase Management:** Coordinates three-phase protocol execution
- **Security Analysis:** Evaluates privacy preservation against various attack scenarios
- **Error Handling:** Manages protocol failures and provides detailed diagnostics

4. Demo Interface (demo_script.py)

Purpose: Provides interactive demonstration of the SMPC protocol capabilities.

Demo Features:

- **Interactive Input:** Allows users to specify company revenues for realistic scenarios
- **Real-time Visualization:** Shows protocol progress through each phase
- **Security Demonstration:** Illustrates privacy preservation properties
- **Testing Interface:** Comprehensive test suite execution and validation

3.4 Protocol Flow

Phase 1: Share Distribution

1. Each party generates polynomial shares of their secret value
2. Shares are distributed to all parties via secure channels
3. Each party receives one share from every other party
4. Parties verify complete share reception before proceeding

Mathematical Process:

- Party i with secret s_i generates polynomial: $f_i(x) = s_i + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1}$
- Share for party j : $(j, f_i(j))$ where $j \in \{1, 2, \dots, n\}$
- All coefficients except s_i are randomly chosen from $\mathbb{Z}_p$

Phase 2: Computation

1. Each party locally sums their received shares
2. addition: $\sum_i f_i(j) = f_sum(j)$ where $f_sum(0) = \sum_i s_i$
3. No individual secrets are ever reconstructed
4. Only sum shares are computed and stored

Key Insight: The sum of shares at each coordinate represents a share of the final sum:

- Sum share at party j : $(j, f_1(j) + f_2(j) + \dots + f_n(j))$
- These sum shares can reconstruct $\sum_i s_i$ but not individual secrets

Phase 3: Threshold Reconstruction

1. Threshold number of parties provide their sum shares
2. Lagrange interpolation reconstructs the final sum
3. Individual secrets remain cryptographically protected
4. Only the aggregate result is revealed

Security Guarantee: With less than t sum shares, the final result is information-theoretically secure.

This local architecture provides a robust foundation for understanding and demonstrating secure multi-party computation principles while maintaining the flexibility to extend to distributed deployments when needed.

4. Implementation Overview

4.1 Core Components

Our implementation consists of several key modules:

4.1.1 Cryptographic Module (`smpc_crypto.py`)

This module implements the fundamental cryptographic operations:

Key Functions:

- `get_prime(bits)`: Generates cryptographically secure primes
- `create_shares(secret, threshold, num_shares, prime)`: Implements Shamir's secret sharing
- `reconstruct_secret(shares, prime)`: Performs Lagrange interpolation

Security Features:

- Uses cryptographically secure random number generation
- Implements proper modular arithmetic
- Validates input parameters to prevent attacks

4.1.2 Party Module (`party.py`)

Implements individual party logic:

Classes:

- `DecentralizedParty`: Main party implementation with autonomous behavior
- `Share`: Data structure for share representation
- `SecureChannel`: Simulates authenticated communication

Capabilities:

- Independent share generation and distribution
- Local computation without revealing secrets
- Status tracking and protocol state management

4.1.3 Controller Module (smpc_controller.py)

Orchestrates the overall protocol:

Classes:

- DecentralizedSMPCController: Coordinates protocol execution
- SMPCController: Legacy compatibility layer

Features:

- Phase-by-phase protocol coordination
- Error handling and recovery
- Performance metrics collection

4.2 Implementation Highlights

4.2.1 Field Operations

All arithmetic is performed in a finite field $\mathbb{F}_p$ where p is a large prime:

```
def _mod_inverse(a: int, m: int) -> int:
```

```
    """Calculate modular inverse using extended Euclidean algorithm"""
```

```
    def extended_gcd(a, b):
```

```
        if a == 0:
```

```
            return b, 0, 1
```

```
        gcd, x1, y1 = extended_gcd(b % a, a)
```

```
        x = y1 - (b // a) * x1
```

```
        y = x1
```

```
        return gcd, x, y
```

```
    if a < 0:
```

```
        a = (a % m + m) % m
```

```
    gcd, x, _ = extended_gcd(a, m)
```

```
    if gcd != 1:
```

```
        raise ValueError('Modular inverse does not exist')
```

```
    return x % m
```

This function calculates the modular inverse of a number using the extended Euclidean algorithm. The modular inverse of a modulo m is a number x such that $(a * x) \% m = 1$. The function first uses a nested helper function to find the greatest common divisor and the coefficients needed for the inverse calculation. It handles negative inputs by normalizing them, checks that the inverse exists (gcd must equal 1), and returns the positive modular inverse.

4.2.2 Polynomial Evaluation

Efficient polynomial evaluation using Horner's method:

```
def _evaluate_polynomial(coefficients: List[int], x: int, prime: int) -> int:
```

```
    """Evaluate polynomial using Horner's method"""
```

```
    result = 0
```

```
    for coeff in reversed(coefficients):
```

```
        result = (result * x + coeff) % prime
```

```
    return result
```

function efficiently evaluates a polynomial at a given point using Horner's method, which reduces the number of multiplications needed. Instead of calculating each term separately and summing them, Horner's method rewrites the polynomial in a nested form that can be evaluated from the highest degree term down to the constant term. All arithmetic is performed modulo a prime number to keep the values within a specific range and maintain the mathematical properties needed for secret sharing.

4.2.3 Lagrange Interpolation

Reconstruction using Lagrange interpolation:

```
def _lagrange_interpolation(points: List[Tuple[int, int]], prime: int, x: int = 0) -> int:
```

```
    """Reconstruct secret using Lagrange interpolation"""
```

```
    result = 0
```

```
    n = len(points)
```

```
    for i in range(n):
```

```
        xi, yi = points[i]
```

```
        term = yi
```

```
        for j in range(n):
```

```
            if i != j:
```

```
                xj, _ = points[j]
```

```
                numerator = (x - xj) % prime
```

```
                denominator = (xi - xj) % prime
```

```
                if denominator == 0:
```

```
                    raise ValueError("Duplicate x-values in shares detected")
```

```
                inv_denominator = _mod_inverse(denominator, prime)
```

```
                term = (term * numerator * inv_denominator) % prime
```

```
    result = (result + term) % prime
```

```
    return result
```

This function reconstructs a secret value from a set of coordinate points using Lagrange interpolation, a mathematical technique that finds a unique polynomial passing through given points. For each point, it calculates a Lagrange basis polynomial that equals 1 at that point and 0 at all other points, then combines these weighted by the y-values. The function includes error checking for duplicate x-values and uses modular arithmetic throughout to ensure the reconstruction works correctly within the finite field defined by the prime modulus.

5. Simple Simulation Analysis

5.1 Overview

The simple simulation demonstrates basic SMPC functionality in a controlled local environment. This simulation serves as a proof-of-concept and educational tool for understanding SMPC principles.

5.2 Simulation Setup

5.2.1 Configuration Parameters

Basic simulation parameters

num_parties = 3 # Three participating parties

threshold = 2 # Minimum parties needed for reconstruction

secrets = [100, 200, 300] # Individual party secrets

prime_bits = 512 # Security parameter for field size

5.2.2 Execution Environment

The simple simulation runs entirely in local memory:

- No network communication required
- Immediate execution with minimal latency
- Ideal for educational purposes and protocol verification

5.3 Step-by-Step Execution

5.3.1 Initialization Phase

```
def demonstrate_basic_workflow():
```

```
    """Demonstrate the full basic SMPC workflow"""
```

```
    print("🇮🇹 BASIC SMPC WORKFLOW DEMONSTRATION")
```

```
    # Setup
```

```
    secrets = [100, 200, 300] # Company revenues
```

```
    threshold = 2
```

```
    print(f"Secrets: {secrets}")
```

```
    print(f"Expected sum: {sum(secrets)}")
```

```
    # Initialize SMPC system
```

```
    smpc = DecentralizedSMPCController(secrets, threshold)
```

Analysis: This phase establishes the computational environment and verifies that all cryptographic parameters are properly configured.

5.3.2 Share Generation and Distribution

```
# Phase 1: Create shares for each secret
print("Phase 1: Creating and distributing shares")
share_map = smpc.create_shares_for_parties(secrets)

for party in smpc.parties:
    shares = share_map[party.id]
    print(f"Party {party.id}: {[share.name for share in shares]}")
```

Technical Details:

- Each party's secret is split into n shares using a random polynomial
- Shares are distributed such that each party receives one share per secret
- The threshold property ensures any t parties can reconstruct the result

Security Analysis: At this stage, no individual secret can be reconstructed from fewer than t shares, providing perfect secrecy.

5.3.3 Local Computation Phase

```
# Phase 2: Local computation without secret reconstruction
print("Phase 2: Computing party sums")
partial_results = smpc.request_parties_to_compute_results(share_map)

for party_id, value in partial_results.items():
    print(f"Party {party_id}: {str(value)[:5]}...")
```

Key Innovation: Parties compute on encrypted shares without ever reconstructing individual secrets. This is achieved through the properties of Shamir's scheme:

If party i holds shares $(s_j^{(i)})$ for $j = 1, 2, \dots, k$, then:

$$\text{sum_share}_i = \sum_j s_j^{(i)} \pmod{p}$$

These sum_shares represent shares of the final sum $\sum_j \text{secret}_j$.

5.3.4 Result Reconstruction

```
# Phase 3: Reconstruct only the final sum
print("Phase 3: Reconstructing final sum")
final_result = smpc.reconstruct_final_result(partial_results)
```

```
print(f"Final result: {final_result}")
print(f"Expected: {sum(secrets)}")
```

Mathematical Process: Using Lagrange interpolation on threshold shares:

$$\text{result} = \sum_i \text{sum_share}_i \cdot \Lambda_i \pmod{p}$$
where Λ_i are Lagrange coefficients.

5.4 Simulation Results

5.4.1 Expected Output

 BASIC SMPC WORKFLOW DEMONSTRATION

Secrets: [100, 200, 300]

Expected sum: 600

Phase 1: Creating and distributing shares

Party 1: ['A_1', 'A_2', 'A_3']

Party 2: ['B_1', 'B_2', 'B_3']

Party 3: ['C_1', 'C_2', 'C_3']

Phase 2: Computing party sums

Party 1: 12345...

Party 2: 67890...

Party 3: 54321...

Phase 3: Reconstructing final sum

Final result: 600

Expected: 600

5.4.2 Security Verification

The simulation demonstrates several key security properties:

1. **Input Privacy:** Individual secrets (100, 200, 300) are never revealed
2. **Computation Correctness:** Final result (600) is mathematically accurate
3. **Threshold Security:** Any 2 of 3 parties can reconstruct the result
4. **Secrecy:** Single party cannot determine any individual input

5.5 Educational Value

The simple simulation effectively demonstrates:

5.5.1 Cryptographic Concepts

- Secret sharing and threshold cryptography
- computation properties
- Polynomial interpolation mathematics

5.5.2 Protocol Understanding

- Multi-phase execution model
- Distributed computation without central authority
- Privacy-preserving collaborative computation

6. Servers Simulation Analysis

6.1 Overview

The servers simulation extends the basic concept to a more realistic distributed environment, showcasing practical deployment considerations and real-world applicability.

6.2 Enhanced Features

HTTP-Based Communication: Parties communicate via RESTful APIs, simulating real network conditions.

Asynchronous Processing: Non-blocking operations handle network latency and timing variations.

Fault Tolerance: Robust error handling and retry mechanisms for network failures.

Monitoring and Diagnostics: Real-time status tracking and health monitoring.

6.3 Implementation Details

6.3.1 Party Server Architecture

Each party runs as an independent HTTP server:

```
class DistributedPartyServer:
    """HTTP server for distributed SMPC party"""

    def __init__(self, config: PartyConfig):
        self.config = config
        self.app = Flask(f"SMPC_Party_{config.party_id}")

        # Protocol state
        self.my_secret_shares = []
        self.received_shares = {}
        self.sum_share = None

        # Setup API routes
        self._setup_routes()
```

```

def _setup_routes(self):
    """Setup HTTP API endpoints"""

    @self.app.route('/api/v1/create_shares', methods=['POST'])
    def create_shares():
        # Implementation details...

    @self.app.route('/api/v1/receive_share', methods=['POST'])
    def receive_share():
        # Implementation details...

```

6.3.2 Secure Communication Protocol

API Endpoints:

- POST /api/v1/create_shares: Trigger share generation and distribution
- POST /api/v1/receive_share: Accept share from another party
- POST /api/v1/compute_sum: Perform local sum computation
- GET /api/v1/get_sum_share: Provide share for final reconstruction
- GET /api/v1/status: Report party status and readiness

Message Format:

```

{
  "share": {
    "value": 1234567890,
    "x_coord": 1,
    "secret_owner_id": 2,
    "share_holder_id": 1
  },
  "sender_id": 2,
  "timestamp": 1703123456.789

```

```
}
```

6.3.3 Coordination Protocol

The coordinator orchestrates the protocol execution:

```
class DistributedSMPCCoordinator:
    """Coordinates distributed SMPC execution"""

    def run_distributed_protocol(self):
        """Execute complete distributed protocol"""

        # Phase 1: Distributed share creation
        if not self._phase1_distribute_shares():
            return 0, False

        # Phase 2: Sum computation
        if not self._phase2_compute_sums():
            return 0, False

        # Phase 3: Result reconstruction
        final_sum = self._phase3_reconstruct_sum()

        return final_sum, True
```

6.4 Execution Scenario

6.4.1 Multi-Company Financial Consortium

Scenario: Four financial institutions want to compute total loan defaults without revealing individual bank data.

Participants:

- **Bank A:** \$3,500,000 in defaults (Secret)
- **Bank B:** \$2,500,000 in defaults (Secret)
- **Bank C:** \$1,500,000 in defaults (Secret)

Security Requirements:

- Individual bank data must remain confidential
- Only aggregate total should be revealed

6.4.2 Protocol Execution

Step 1: Initialization

Coordinator: Initializing 3-party SMPC with threshold=2

Bank A Server: Starting on localhost:5001

Bank B Server: Starting on localhost:5002

Bank C Server: Starting on localhost:5003

Step 2: Health Verification

Coordinator: Checking party health...

✅ Bank A (localhost:5001): healthy

✅ Bank B (localhost:5002): healthy

✅ Bank C (localhost:5003): healthy

Step 3: Share Distribution

Bank A: Creating 3 shares of secret \$3,500,000

Bank A: Distributing shares to all parties via HTTP

Bank B: Creating 3 shares of secret \$2,500,000

Bank B: Distributing shares to all parties via HTTP

Bank C: Creating 3 shares of secret \$1,500,000

Bank C: Distributing shares to all parties via HTTP

Coordinator: All parties successfully distributed shares

Coordinator: Waiting for share reception confirmation...

Coordinator: All parties received all shares ✅

Step 4: Local Computation

Bank A: Computing sum share from received shares...

Bank A: Sum computation successful

Bank B: Computing sum share from received shares...

Bank B: Sum computation successful

Bank C: Computing sum share from received shares...

Bank C: Sum computation successful

Step 5: Final Reconstruction

Coordinator: Collecting sum shares from 3 parties (threshold)

Coordinator: Using shares from Bank A, Bank B, Bank C

Coordinator: Reconstructing final sum...

DISTRIBUTED SMPC COMPLETED SUCCESSFULLY!

Final Result: 7,500,000\$

Execution Time: 36.65s

Parties Used: 3

Threshold Security: 2

7. Code Structure and Analysis

7.1 Module Architecture

Our implementation follows a layered architecture with clear separation of concerns:

7.1.1 Core Cryptographic Layer

File: smpc_crypto.py

Purpose: Implements fundamental cryptographic primitives

Key Components:

Prime generation for finite field operations

def get_prime(bits: int = 512) -> int:

"""Generate cryptographically secure prime"""

return number.getPrime(bits)

Secret sharing implementation

def create_shares(secret: int, threshold: int, num_shares: int, prime: int):

"""Split secret using Shamir's scheme"""

Generate random polynomial coefficients

coefficients = [secret] + [secrets.randbelow(prime - 1) + 1

```

        for _ in range(threshold - 1)]

# Evaluate polynomial at distinct points
return [(i, _evaluate_polynomial(coefficients, i, prime))
        for i in range(1, num_shares + 1)]

# Secret reconstruction
def reconstruct_secret(shares: List[Tuple[int, int]], prime: int) -> int:
    """Reconstruct secret using Lagrange interpolation"""
    return _lagrange_interpolation(shares, prime)

```

7.1.2 Party Logic Layer

File: party.py

Purpose: Implements individual party behavior and state management

Key Classes:

@dataclass

class Share:

```

    """Represents a cryptographic share"""
    value: int          # Share value in finite field
    x_coord: int         # X-coordinate for interpolation
    secret_owner_id: int # Which party's secret this represents
    share_holder_id: int # Which party holds this share

```

class DecentralizedParty:

```

    """Autonomous party in SMPC protocol"""

    def __init__(self, party_id, secret_value, threshold, total_parties,
                  secure_channel, prime):
        self.party_id = party_id

```

```

self.secret_value = secret_value % prime # Normalize to field
self.received_shares = {} # Shares from other parties
self.sum_share = None # Computed sum share

def phase1_create_and_distribute_shares(self):
    """Create and distribute shares of own secret"""
    raw_shares = crypto.create_shares(
        self.secret_value, self.threshold,
        self.total_parties, self.prime
    )

    # Convert to Share objects and distribute
    for x_coord, share_value in raw_shares:
        share = Share(share_value, x_coord, self.party_id, x_coord)
        self._send_share_to_party(share)

def phase2_compute_sum_share(self):
    """Compute share of final sum without reconstruction"""
    sum_value = 0
    for owner_id, share in self.received_shares.items():
        sum_value = crypto.add_shares([sum_value, share.value], self.prime)

    self.sum_share = Share(sum_value, self.party_id, 0, self.party_id)

```

Design Principles:

- **Encapsulation:** Each party manages its own state independently
- **Security:** Sensitive operations are clearly isolated
- **Modularity:** Phases are separated for clear protocol flow

7.1.3 Protocol Coordination Layer

File: smpc_controller.py

Purpose: Orchestrates protocol execution and manages party coordination

Key Classes:


```

class DecentralizedSMPCController:
    """Coordinates decentralized SMPC protocol execution"""

    def __init__(self, secrets: List[int], threshold: int, prime: Optional[int] =
None):
        # Input validation
        if threshold > len(secrets):
            raise ValueError("Threshold cannot exceed number of parties")

        self.secrets = [int(s) for s in secrets] # Type validation
        self.threshold = threshold
        self.prime = prime if prime else crypto.get_prime(512)

        # Create autonomous parties
        self.parties = [
            DecentralizedParty(i+1, secret, threshold, len(secrets),
                               self.secure_channel, self.prime)
            for i, secret in enumerate(secrets)
        ]

    def run_secure_computation(self) -> Tuple[int, bool]:
        """Execute complete SMPC protocol"""
        try:
            # Phase 1: Decentralized share distribution
            if not self._phase1_distributed_sharing():
                return 0, False

            # Phase 2: Local sum computation
            if not self._phase2_secure_sum_computation():
                return 0, False

            # Phase 3: Final reconstruction

```

```

        final_sum = self._phase3_sum_reconstruction()

        # Verify correctness
        success = self._verify_correctness(final_sum)

        return final_sum, success

    except Exception as e:
        self.logger.error(f"Protocol failed: {e}")
        return 0, False

```

Coordination Features:

- **Error Handling:** Comprehensive exception management
- **State Tracking:** Monitor protocol progress across all parties
- **Verification:** Automatic correctness checking

7.2 Communication Infrastructure

7.2.1 Secure Channel Simulation

File: party.py (SecureChannel class)

Purpose: Simulates authenticated, encrypted communication

class SecureChannel:

```

    """Simulates secure point-to-point communication"""

```

```

    def __init__(self):

```

```

        self.message_queues = defaultdict(queue.Queue)

```

```

        self.message_log = []

```

```

        self.total_messages = 0

```

```

        self.lock = threading.Lock()

```

```

    def send_message(self, message: SecureMessage):

```

```

        """Send message through secure channel"""

```

```

        with self.lock:

```

```

            self.message_queues[message.receiver_id].put(message)

```

```

            self.message_log.append(message)

```

```

        self.total_messages += 1

def receive_messages(self, party_id: int) -> List[SecureMessage]:
    """Receive all pending messages for a party"""
    messages = []
    queue_obj = self.message_queues[party_id]

    while not queue_obj.empty():
        try:
            message = queue_obj.get_nowait()
            messages.append(message)
        except queue.Empty:
            break

    return messages

```

7.2.2 Distributed Communication (HTTP)

File: distributed_party_server.py

Purpose: Implements HTTP-based communication for distributed deployment

class DistributedPartyServer:

```

    """HTTP server for distributed SMPC party"""

    def _setup_routes(self):
        """Setup RESTful API endpoints"""

        @self.app.route('/api/v1/receive_share', methods=['POST'])
        def receive_share():
            try:
                data = request.get_json()
                share_data = data.get('share')

                # Validate and deserialize share
                share = DistributedShare.from_dict(share_data)

```

```

# Store received share
self.received_shares[share.secret_owner_id] = share

# Check completion status
expected_senders = set(range(1, self.config.total_parties + 1))
received_senders = set(self.received_shares.keys())
self.all_shares_received = (expected_senders ==
received_senders)

return jsonify({
    "success": True,
    "all_shares_received": self.all_shares_received,
    "received_from": list(received_senders)
})

except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

```

API Design:

- **RESTful Architecture:** Standard HTTP methods and status codes
- **JSON Serialization:** Structured data exchange format
- **Error Handling:** Comprehensive exception management

7.3 Testing Infrastructure

File: test_smpc.py

Purpose: Extensive testing covering edge cases and failure scenarios

```
class TestFailureCasesInvalidInput(unittest.TestCase):
```

```
    """Test system behavior with invalid inputs"""
```

```
    def test_threshold_too_high(self):
```

```
        """Test when threshold > number of parties"""
```

```
        secrets = [100, 200, 300]
```

```
        threshold = 5 # More than 3 parties
```

```
        with self.assertRaises(ValueError):
```

```
            DecentralizedSMPCController(secrets, threshold)
```

```
    def test_empty_secrets(self):
```

```
        """Test with empty secrets list"""
```

```
        with self.assertRaises(ValueError):
```

```
            DecentralizedSMPCController([], 1)
```

```
class TestPerformanceStressTests(unittest.TestCase):
```

```
    """Test system performance under stress"""
```

```
    def test_large_number_of_parties(self):
```

```
        """Test with many parties"""
```

```
        num_parties = 10
```

```
        secrets = [random.randint(1000, 9999) for _ in range(num_parties)]
```

```
        threshold = num_parties // 2 + 1
```

```
        start_time = time.time()
```

```
        controller = DecentralizedSMPCController(secrets, threshold)
```

```
        result, success = controller.run_secure_computation()
```

```
execution_time = time.time() - start_time
```

```
self.assertTrue(success)
```

```
self.assertLess(execution_time, 30.0) # Performance requirement
```

The test_smpc.py file implements a comprehensive testing framework designed to validate the robustness, security, and performance characteristics of the SMPC system. Unlike basic unit tests that verify normal operation, this testing specifically targets edge cases, failure scenarios, and stress conditions to ensure the system behaves correctly under all circumstances.

8. Future Work and Conclusions

8.1 Summary of Achievements

This technical report has presented a comprehensive implementation of Secure Multi-Party Computation (SMPC) based on Shamir's Secret Sharing scheme, demonstrating the practical feasibility of privacy-preserving collaborative computation. The project successfully achieved all primary objectives established in the introduction:

Cryptographic Foundation: We implemented a robust Shamir's Secret Sharing system with configurable (t,n) -threshold parameters, providing information-theoretic security guarantees. The implementation includes proper finite field arithmetic, secure polynomial evaluation, and Lagrange interpolation for secret reconstruction.

Complete SMPC Protocol: The system delivers a fully functional multi-party computation framework that enables parties to collaboratively compute aggregate functions while maintaining perfect input privacy. The modular architecture supports both local simulation and distributed deployment scenarios.

Security Validation: Through extensive testing and theoretical analysis, we demonstrated that the system maintains privacy guarantees even when up to $t-1$ parties collude. The implementation provides verifiable security properties including input confidentiality, correctness of computation, and threshold-based protection.

Practical Implementation: The project includes two deployment models - a local simulation for educational purposes and a distributed HTTP-based system for real-world applications. Both implementations showcase the versatility and practical applicability of SMPC in various computational environments.

8.2 Key Contributions

The implementation makes several significant contributions to the field of secure computation:

Educational Framework: The local simulation provides an accessible platform for understanding SMPC principles, making complex cryptographic concepts comprehensible through interactive demonstrations and comprehensive documentation.

Comprehensive Testing: The extensive test suite covering edge cases, performance stress tests, and security validation provides a robust foundation for further development and deployment confidence.

Modular Design: The layered architecture with clear separation between cryptographic primitives, party logic, and protocol coordination facilitates future extensions and modifications.

8.3 Future Work

While the current implementation provides a solid foundation for SMPC applications, several areas present opportunities for significant enhancement and research:

8.3.1 Cryptographic Enhancements

Advanced Computation Support: Extend beyond additive functions to support multiplication, comparison operations, and complex mathematical functions. This would require implementing more sophisticated protocols such as BGW or GMW to handle arbitrary circuit evaluation.

Optimization Techniques: Implement advanced optimization strategies including:

- Packed secret sharing for parallel computation
- Preprocessing techniques to reduce online computation costs
- Batch processing for multiple simultaneous computations
- Optimized field arithmetic using specialized libraries

8.3.2 Protocol Extensions

Malicious Security: The current implementation assumes semi-honest adversaries. Future work should incorporate zero-knowledge proofs and commitment schemes to provide security against malicious parties who may deviate from the protocol.

Dynamic Party Management: Implement protocols supporting dynamic addition and removal of parties during computation, enabling flexible consortium membership and fault recovery mechanisms.

Verifiable Computation: Add cryptographic proofs allowing parties to verify the correctness of computations without revealing private inputs, enhancing trust in distributed environments.

8.4 Conclusions

This project has successfully demonstrated the practical viability of Secure Multi-Party Computation for privacy-preserving collaborative analytics. The implementation provides both theoretical validation and practical deployment capabilities, bridging the gap between academic research and real-world applications.

The significance of this work extends beyond the technical implementation. As data privacy regulations become increasingly stringent and organizations

seek collaborative insights without compromising sensitive information, SMPC emerges as a critical technology enabler. Our implementation provides a foundation for addressing these challenges in domains ranging from healthcare and finance to government and corporate analytics.

The modular architecture and comprehensive testing framework ensure that this implementation can serve as both an educational tool for understanding SMPC principles and a production-ready platform for deploying privacy-preserving applications. The clear separation of concerns between cryptographic primitives, party logic, and protocol coordination facilitates future enhancements and customizations.

Impact on Privacy-Preserving Computing: This implementation contributes to the growing ecosystem of privacy-preserving technologies by providing a practical, well-documented reference implementation that demonstrates the feasibility of secure multi-party computation in real-world scenarios.

Educational Value: Comprehensive documentation, interactive demonstrations, and extensive testing provide valuable resources for researchers, students, and practitioners seeking to understand and implement SMPC protocols.

Foundation for Innovation: Extensible architecture and modular design create opportunities for future research and development in areas such as malicious security, advanced computation support, and large-scale deployment optimization.

As organizations increasingly recognize the value of collaborative computation while maintaining data privacy, implementations like this will play a crucial role in enabling secure, privacy-preserving analytics across organizational boundaries. The future of data collaboration lies in technologies that can provide both computational utility and privacy protection, and this SMPC implementation represents a significant step toward that vision.

The successful completion of this project demonstrates that secure multi-party computation has evolved from a theoretical concept to a practical technology ready for deployment in privacy-sensitive applications. With continued development and optimization, SMPC will become an essential component of the privacy-preserving computing landscape, enable new forms of collaboration while maintain the highest standards of data protection.